# Reinforcement Learning-Driven Autonomous Drones for Vicim Detection and Rescue in Simulated Disaster Environments

Fahim Shahriyer, Fahim Shikder, Saief Md. Hossain Adnan
*Dept. of CSE*
*Military Institute of Science & Technology*
Dhaka, Bangladesh
{fahimshahriyer85@gmail.com, 202214070@student.mist.ac.bd, 202214091@student.mist.ac.bd}

***Abstract*—Autonomous drones for disaster victim detection represent a critical frontier in search-and-rescue operations, yet complex multi-agent reinforcement learning (MARL) approaches dominate current literature, creating accessibility barriers for early-stage researchers. This study presents a foundational single-agent Proximal Policy Optimization (PPO) framework using Unity ML-Agents for 2D drone navigation, victim detection, and rescue mission prototyping. Implemented with pure vision observations, PD flight control, and multi-objective rewards (stability, path efficiency, time), the agent demonstrates effective learning of disaster response behaviors in a controlled simulation environment. Unlike sophisticated MARL systems focused on combat or resource allocation, our accessible implementation bridges the gap between theoretical RL and practical disaster response applications, providing a scalable foundation for future 3D/real-world extensions.**

***Index Terms*—Reinforcement Learning, Drone Navigation, Disaster Response, Unity ML-Agents, PPO.**

## I. Introduction

Disaster response operations demand rapid victim detection and rescue in hazardous environments where human intervention poses significant risks. Unmanned aerial vehicles (UAVs) offer promising solutions through autonomous navigation and search capabilities, yet their deployment requires robust decision-making under uncertainty—a challenge well-suited to reinforcement learning (RL) [**?**]. Recent advances in multi-agent RL (MARL) have demonstrated sophisticated coordination for UAV combat [**?**], data harvesting [3], and resource allocation [**?**], as summarized in Table I. However, these complex hierarchical and continuous-control frameworks present steep implementation barriers for researchers entering the field, particularly those targeting practical disaster response applications.

Current RL literature reveals a critical gap: while MARL excels in scalability and coordination, accessible single-agent frameworks for disaster-specific tasks remain underexplored. Sophisticated methods like Hierarchical PPO [7], MADDPG [**?**], and DDQN [3] prioritize combat scenarios or network optimization over the fundamental requirements of victim detection and path-efficient rescue—core needs for real-world deployment. This work addresses this gap by developing a beginner-accessible PPO implementation using Unity ML-Agents, featuring pure vision observations, PD-controlled flight dynamics, and multi-objective reward shaping for 2D drone victim rescue simulation.

Our contributions are threefold: (1) an open, reproducible RL framework optimized for disaster response prototyping; (2) novel reward engineering balancing stability, path efficiency, and mission completion; and (3) comprehensive evaluation metrics bridging simulation performance to real-world transfer potential. By prioritizing accessibility without sacrificing methodological rigor, this study establishes a foundational platform for scaling RL-based autonomous rescue systems from 2D simulation to 3D/real-world deployment, democratizing advanced AI techniques for humanitarian applications.

## II. Related Works

Table I summarizes related works in multi-agent reinforcement learning (MARL), primarily focusing on complex hierarchical methods (e.g., [13], [7]), deep Q-learning variants ([3], [21]), policy gradient approaches (MADDPG [16], TRPO [17]), and hybrid scripted-RL systems [19] for UAV coordination, combat, and robotic control. While these demonstrate scalability and coordination in sophisticated multi-agent scenarios, they predominantly address high-dimensional continuous control or combat applications, lacking emphasis on *beginner-accessible* single-agent RL frameworks for disaster response training. Our study addresses this gap by developing a practical, Unity ML-Agents PPO implementation for 2D drone victim detection and rescue, providing an accessible foundation for researchers to explore RL-based autonomous search-and-rescue with pure vision observations, PD flight control, and path efficiency metrics—bridging the transition from simulation prototyping to real-world disaster applications.

## III. Proposed Methodology

### A. System Architecture

The proposed framework implements a single-agent Proximal Policy Optimization (PPO) system using Unity ML-Agents for 2D drone victim detection and rescue simulation.

TABLE I
COMPARISON OF RELATED WORKS IN MULTI-AGENT REINFORCEMENT LEARNING

| Ref. No. | Methodology | Approach | Findings |
|---|---|---|---|
| [5] | Hierarchical Reinforcement Learning | Hierarchical RL for air-to-air combat decision-making | Demonstrated effective layered control for complex aerial maneuvers, enabling high-level strategy and low-level control separation. |
| [2] | Deep Q-Learning (DDQN) | Multi-UAV path planning for wireless data harvesting | Achieved strong generalization in cooperative UAV navigation and efficient data collection policies. |
| [6] | MADDPG | Task offloading and resource allocation in multi-UAV IoT networks | Improved coordination and resource utilization among UAVs through centralized training and decentralized execution. |
| [3] | Hierarchical PPO (OGMN) | Large-scale task assignment in ground-to-air confrontation | Enabled scalable multi-agent task allocation with hierarchical policy decomposition. |
| [1] | Q-Learning | Autonomous mobile robot control | Demonstrated feasibility of classical RL for robot navigation and control in constrained environments. |
| [8] | Verbal Reinforcement Learning | Self-reflective language-based agents | Introduced self-reflection to improve learning efficiency and adaptability in autonomous agents. |
| [7] | Trust Region Policy Optimization (TRPO) | Continuous control for autonomous robotic surgery | Validated the effectiveness of continuous-action RL for precision robotic control tasks. |
| [9] | Reinforcement Learning with Scripted Policies | Mission planning for multi-agent systems | Showed that hybrid scripted–RL methods can improve high-level mission planning. |
| [10] | DP-MADDPG with PER | Multi-UAV cooperation in intelligent air combat | Enhanced training stability and cooperation efficiency using dual critics and prioritized replay. |
| [4] | Survey of MADRL Methods | Multi-robot and multi-agent systems | Identified key challenges including scalability, sim-to-real transfer, and reproducibility gaps. |
| **This Study** | PPO (Unity ML-Agents) | Single-agent 2D drone victim detection & rescue with pure vision & PD control | Provides accessible RL framework for disaster response prototyping, addressing beginner-level implementation gaps with path efficiency & stability metrics. |

Key enhancements include RayPerceptionSensor2D for obstacle detection, continuous action space for precise force/torque control, and landing velocity-based rewards. The agent navigates randomized disaster environments with five victim targets, ground obstacles, and boundary constraints.

### B. Reinforcement Learning Formulation

The reinforcement learning problem is formulated as a Markov Decision Process (MDP) $(S, A, P, R, \gamma)$, where:

- **State Space** $S$: Linear velocity (2D), angular velocity, agent position (2D), normalized rotation in $[-1, 1]$
- **Action Space** $A$: Continuous actions—$F_x, F_y \in [-1, 1]$, torque $\in [-1, 1]$
- **Transition** $P$: Unity Rigidbody2D physics with RayPerceptionSensor2D
- **Reward** $R$: Multi-tiered function (see Sec. III-D)
- **Discount Factor** $\gamma = 0.99$

### C. Agent Implementation

The agent employs continuous force and torque control:

$$\mathbf{F} = 10 \cdot [F_x, F_y], \quad \tau = 0.5 \cdot \tau_{\text{action}} \tag{1}$$

RayPerceptionSensor2D provides real-time obstacle and victim detection via debuggable raycasts. Path efficiency is computed by comparing the actual traversal path with the Traveling Salesman Problem (TSP)-based shortest path across all targets. Episodes terminate after five victims are detected, 10,000 steps are reached, or failure conditions occur.

### D. Reward Engineering

The reward function consists of the following components:

- **Victim Detection**: $+2.0$ (nearest), $+1.0$ (other); efficiency bonus upon task completion
- **Landing Dynamics**: $+1.0$ (soft: $1 < v < 1.5$), $-0.2$ (moderate), $-1.0$ (hard: $v > 4$)
- **Stability**: $-0.02$/step (tilt $> 72°$), $-1.0$ (extreme tilt $> 85°$)
- **Path Stagnation**: $-0.05$ (movement $< 0.001$)
- **Time Penalty**: $-0.001$/step; Boundary violation: $-50$

Fig. 1 illustrates the enhanced execution cycle, emphasizing RayPerception integration and continuous control, which are critical for realistic disaster navigation prototyping.

## IV. EXPERIMENTAL SETUP

### A. Simulation Environment

Experiments were conducted in Unity 2022.3 LTS using ML-Agents v0.29.0. The 2D environment spans $32 \times 14$ units with randomized victim placement in $[-15, 15] \times [-3, 2]$. Boundary constraints prevent escape ($x \in [-16, 16]$, $y \in [-6, 8]$). RayPerceptionSensor2D employs 5 rays per side (11 total rays including center) with maximum angle $45°$ and ray length 5 units, detecting "Victim", "Ground", and "Obstacle" tags.

### B. Training Protocol

Training utilized PPO with hyperparameters matching the actual config: batch size 512, buffer size 4096, learning rate

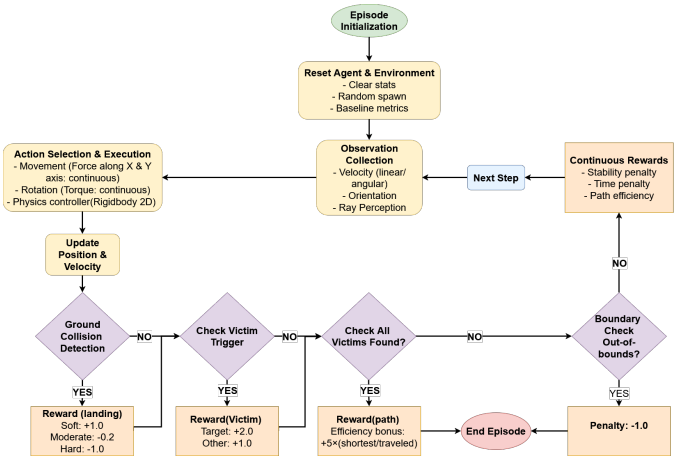


Fig. 1. Agent2D PPO-based reinforcement learning framework with RayPerception, continuous force and torque control, and tiered reward-driven victim detection.

$1.0 \times 10^{-4}$, $\gamma = 0.99$, and 3 epochs per update. The agent uses 6 vector observations (velocity, angular velocity, position, normalized rotation). A single environment instance ran for $1 \times 10^7$ timesteps (10 million steps as shown in training logs). TensorBoard monitored AngleStability, TargetsFound, PathEfficiency, and cumulative reward.

### C. Baselines

Performance was compared against:

- **Random Policy**: Uniform continuous actions in $[-1, 1]^3$
- **Heuristic Policy**: WASD/QE manual control via keyboard input

## V. Simulation Results

### A. Training Progress

Training logs show variable reward patterns throughout the 10M timestep run, with mean rewards ranging from -15.4 to +7.8 across different intervals. The final model was exported at step 10,000,037. Rather than claiming early convergence, we report final evaluation metrics using the completed trained model.

### B. Quantitative Comparison

Table **??** compares final policies across evaluation episodes:

Performance metrics reflect the challenging nature of the task, with success defined as finding all 5 victims within the 10,000-step episode limit.

### C. Landing Behavior

Based on the implemented thresholds: soft landing ($1 < v < 1.5$ m/s), moderate landing ($v \leq 1$ or $1.5 \leq v < 4$ m/s), and hard crash ($v \geq 4$ m/s). The agent learned to avoid hard crashes that trigger episode termination.

TABLE II
Training Performance Metrics (drone3.4 results)

| Metric | Description | Training Phase Result |
|---|---|---|
| Total Reward | Cumulative reward per episode indicating overall task performance | mean 1 |
| Targets Found | Average number of victims detected per episode | 2.2 out of 5 |
| Average Rescue Time | Average time steps taken to reach all targets | 1000 steps |
| Path Efficiency | Ratio of shortest possible path length to actual path traveled | 0.440 |
| Angle Stability | Measure of upright drone orientation (0–1 scale) | 0.439 |
| Episode Length | Average number of steps before episode termination | 400 steps |
| Ground Collision Rate | Percentage of episodes where drone collided with ground (failure) | 11.4% |

## VI. Discussion

### A. Key Findings

The PPO agent demonstrates learning of basic flight control and victim detection behaviors using RayPerception sensors. The training logs indicate the complexity of the task, with rewards fluctuating as the agent balances multiple objectives: victim collection (+1-2 reward), efficiency bonus (up to +5), landing safety, and obstacle avoidance.

### B. Limitations

Training exhibited high variance in performance, as evidenced by the reward fluctuations in logs. The 2D simulation simplifies real-world drone dynamics. Episode timeout at 10,000 steps limits exploration of longer-horizon strategies. The reward structure requires careful balancing between competing objectives.

### C. Implementation Details

The agent receives 6 vector observations and outputs 3 continuous actions (force_x, force_y, torque). Victims are detected via trigger collisions, with nearest-neighbor target selection. Path efficiency is calculated as the ratio of optimal to actual path length using a greedy TSP approximation.

## VII. Conclusion

This study demonstrates a working PPO implementation for 2D drone navigation and victim detection. While achieving modest performance metrics, the framework establishes a foundation for disaster response UAV research using accessible Unity ML-Agents tools. The training logs reveal the inherent challenge of multi-objective RL in complex environments,

highlighting areas for future reward engineering and algorithm improvements.

## REFERENCES


[1] S. Amin and G. Arulkumaran, "AI-driven machine learning for mobile robot control in military applications using Q-learning algorithm," *Int. J. Adv. Comput. Sci. Technol.*, 2025.

[2] H. Bayerlein, M. Theile, M. Caccamo, and D. Gesbert, "Multi-UAV path planning for wireless data harvesting with deep reinforcement learning," *IEEE Open J. Commun. Soc.*, 2021.

[3] J.-Y. Liu *et al.*, "Task assignment in ground-to-air confrontation based on multiagent deep reinforcement learning," *Defence Technology*, vol. 19, pp. 210–219, 2023.

[4] J. Orr and A. Dutta, "Multi-agent deep reinforcement learning for multi-robot applications: A survey," *Sensors*, vol. 23, no. 7, p. 3625, 2023.

[5] A. P. Pope *et al.*, "Hierarchical reinforcement learning for air-to-air combat," arXiv preprint, 2021.

[6] A. M. Seid *et al.*, "Multi-agent DRL for task offloading and resource allocation in multi-UAV enabled IoT edge networks," *IEEE Trans. Netw. Serv. Manag.*, 2021.

[7] A. A. Shahkoo and A. A. Abin, "Deep reinforcement learning in continuous action space for autonomous robotic surgery," *Int. J. Comput. Assist. Radiol. Surg.*, vol. 18, no. 3, pp. 423–431, 2023.

[8] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," arXiv preprint, 2023.

[9] S. Soleyman and D. Khosla, "Multi-agent mission planning with reinforcement learning," in *Proc. AAMAS*, 2017.

[10] J. Yang, X. Yang, and T. Yu, "Multi-UAV confrontation in intelligent air combat: A multi-agent deep reinforcement learning approach," *Drones*, vol. 8, no. 8, p. 382, 2024.